

BÀI TẬP 1

Nhóm 5

Trần Anh Minh-21000242

Lê Thị Hương-21000684

Lê Quốc Khánh-21002466

Bài tập 1:

1. $f(n) = O(f(n)^2)$

- **TH1:** $\lim_{n \rightarrow \infty} f(n) = 0$

Khi đó $\forall \varepsilon > 0$ cho trước, luôn tồn tại $n_0(\varepsilon) \in \mathbb{N}^*$ sao cho $\forall n > n_0$ ta có

$$|f(n) - 0| = f(n) < \varepsilon.$$

Ta có $\forall c > 0$ luôn tồn tại $\varepsilon > 0$ và $n_0(\varepsilon)$ sao cho

$$\begin{aligned} f(n) < \varepsilon < \frac{1}{c} \quad \forall n > n_0 \\ \Rightarrow c \cdot f(n) < 1 \\ \Leftrightarrow f(n) > c \cdot f(n)^2 \end{aligned}$$

$$\Rightarrow f(n) \neq O(f(n)^2).$$

Ví dụ: $f(n) = \frac{1}{n}$, ta có $\frac{1}{n} \neq O\left(\frac{1}{n^2}\right)$.

- **TH2:** $\lim_{n \rightarrow \infty} f(n) = L > 0$

Khi đó $\forall \varepsilon > 0$ cho trước, luôn tồn tại $n_0(\varepsilon) \in \mathbb{N}^*$ sao cho $\forall n > n_0$ ta có

$$|f(n) - L| < \varepsilon.$$

$$\Rightarrow L - f(n) < |f(n) - L| < \varepsilon \Rightarrow f(n) > L - \varepsilon.$$

Ta chọn $\varepsilon = \varepsilon_0 < L \Rightarrow f(n) > L - \varepsilon_0 > 0$

Ta thấy luôn tồn tại $c = \frac{1}{L - \varepsilon_0}$ và $n_0 \in \mathbb{N}^*$ sao cho

$$\begin{aligned} f(n) > L - \varepsilon_0 = \frac{1}{c} \quad \forall n > n_0 \\ \Rightarrow 0 < f(n) < c \cdot f(n)^2 \end{aligned}$$

$$\Rightarrow f(n) = O(f(n)^2).$$

Ví dụ: $f(n) = \frac{1}{n} + 1$, ta có $O\left(\left(\frac{1}{n} + 1\right)^2\right) = O\left(\frac{1}{n^2} + \frac{2}{n} + 1\right) = \frac{1}{n} + 1$.

- **TH3:** $\lim_{n \rightarrow \infty} f(n) = +\infty$

Khi đó $\forall M > 0$ cho trước, luôn tồn tại $n_0(M) \in \mathbb{N}^*$ sao cho $\forall n > n_0$ ta có

$$f(n) > M.$$

Ta thấy luôn tồn tại $c = \frac{1}{M}$ và $n_0 \in \mathbb{N}^*$ sao cho

$$\begin{aligned} f(n) > M > \frac{1}{c} \quad \forall n > n_0 \\ \Rightarrow 0 < f(n) < c \cdot f(n)^2 \end{aligned}$$

$$\Rightarrow f(n) = O(f(n)^2).$$

Ví dụ: $f(n) = n$, ta có $n = O(n^2)$

$\Rightarrow$ Khẳng định này đôi lúc đúng.

$$2. f(n) + g(n) = \Theta(\max(f(n), g(n)))$$

Chọn $c_1 = 1, c_2 = 2, n_0 = 1$ ta có

$$0 < c_1 \cdot \max(f(n), g(n)) \leq f(n) + g(n) \leq c_2 \cdot \max(f(n), g(n)) \quad \forall n > n_0$$

$$\Rightarrow f(n) + g(n) = \Theta(\max(f(n), g(n))).$$

$\Rightarrow$ Khẳng định này luôn đúng.

$$3. f(n) + O(f(n)) = \Theta(f(n))$$

Đặt $g(n) = O(f(n))$ khi đó tồn tại $c > 0$ và $n_0 \in \mathbb{N}^*$ sao cho

$$0 < g(n) \leq c \cdot f(n) \quad \forall n > n_0$$

$$\Leftrightarrow f(n) < f(n) + g(n) < (1 + c)f(n).$$

Chọn $c_1 = 1, c_2 = 1 + c, n_0 = 1$ ta có

$$0 < c_1 \cdot f(n) \leq f(n) + O(f(n)) \leq c_2 \cdot f(n) \quad \forall n > n_0$$

$$\Rightarrow f(n) + O(f(n)) = \Theta(f(n)).$$

$\Rightarrow$ Khẳng định này luôn đúng.

Bài tập 2:

Nhóm 1:

$$f_1(n) = n^{0.999999} \log n = O(n^{0.999999} \log n)$$

$$f_2(n) = 10000000n = O(n)$$

$$f_3(n) = 1.000001^n = O(1.000001^n)$$

$$f_4(n) = n^2 = O(n^2)$$

Các hàm số theo thứ tự tăng dần về độ phức tạp tiệm cận: $f_1(n), f_2(n), f_4(n), f_3(n)$.

Nhóm 2:

$$f_1(n) = 2^{2^{1000000}} = O(1)$$

$$f_2(n) = 2^{1000000n} = O((2^{1000000})^n)$$

$$f_3(n) = \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$$

$$f_4(n) = n\sqrt{n} = O(n^{3/2})$$

Các hàm số theo thứ tự tăng dần về độ phức tạp tiệm cận: $f_1(n), f_4(n), f_3(n), f_2(n)$.

Nhóm 3:

$$f_1(n) = n^{\sqrt{n}} = O(n^{\sqrt{n}})$$

$$f_2(n) = 2^n = O(2^n)$$

$$f_3(n) = n^{10} \cdot 2^{n/2} = O(n^{10} \cdot 2^{n/2})$$

$$f_4(n) = \sum_{i=1}^n (i+1) = \frac{n(n+3)}{2} = O(n^2)$$

Các hàm số theo thứ tự tăng dần về độ phức tạp tiệm cận: $f_4(n), f_3(n), f_2(n), f_1(n)$.

Bài tập 3:

1 Giới thiệu Floodfill sử dụng DFS

Floodfill là một thuật toán được sử dụng rộng rãi trong đồ họa máy tính và bài toán trên lưới (grid-based problems). Nhiệm vụ chính của thuật toán là xác định và "lấp đầy" tất cả các ô có cùng thuộc tính với một ô ban đầu trong một lưới hoặc ma trận. Thuật toán này được ứng dụng nhiều trong việc đổ màu vùng (fill color) trong các phần mềm vẽ, như công cụ paint bucket trong Photoshop.

Mô tả thuật toán

Floodfill bắt đầu từ một ô ban đầu trong lưới, sau đó lan ra các ô lân cận theo phương pháp DFS, kiểm tra xem ô lân cận có cùng thuộc tính hay không (ví dụ: có cùng màu hay không). Nếu có, thuật toán tiếp tục lấp đầy vùng xung quanh nó cho đến khi tất cả các ô có thuộc tính tương tự đều được thay đổi.

Ứng dụng

- **Đổ màu trong đồ họa (Paint Bucket Tool):** Khi người dùng chọn một vùng và nhấp vào công cụ đổ màu, thuật toán floodfill sẽ thay đổi màu của toàn bộ khu vực có màu ban đầu liên tục với điểm người dùng chọn.

Chi tiết thuật toán

- Đầu vào:
 - Lưới grid có các ô với thuộc tính nhất định (màu sắc, giá trị số).
 - Một điểm xuất phát (x, y) .
 - Giá trị cần thay thế `new_color`.
- Đầu ra:
 - Lưới với các ô lân cận có cùng thuộc tính với ô tại (x, y) được thay thế bằng `new_color`.

Triển khai thuật toán

1. Kiểm tra xem ô tại bắt đầu (x, y) có thuộc tính cần thay đổi không.
2. Nếu không, kết thúc thuật toán vì không có gì để thay đổi.
3. Nếu còn ô cần thay đổi, duyệt qua các ô lân cận bằng DFS (theo các hướng trái, phải, lên, xuống hoặc cả các hướng chéo tùy theo yêu cầu).
4. Nếu ô lân cận có cùng thuộc tính, thay đổi thuộc tính của nó thành `new_color` và tiếp tục duyệt qua các ô lân cận khác.
5. Quá trình này tiếp tục cho đến khi tất cả các ô cần thay đổi đã được duyệt và cập nhật.

Mã giả (Pseudocode)

Mã giả thuật toán tô màu theo chiều sâu (DFS Floodfill)

Input: Lưới grid, tọa độ ban đầu (x, y) , màu cũ `old_color`, màu mới `new_color`

Output: Lưới grid sau khi đã tô lại các vùng có màu ban đầu là `old_color` bằng `new_color`

Function DFS_Floodfill(grid, x, y, old_color, new_color):

```
if grid[x][y] ≠ old_color then
    return
grid[x][y] ← new_color
foreach (dx, dy) trong [(1, 0), (-1, 0), (0, 1), (0, -1)] do
    if (x + dx, y + dy) nằm trong phạm vi của grid then
        DFS_Floodfill(grid, x + dx, y + dy, old_color, new_color)
```

2 Maze solving using BFS

Vấn đề

Một bài toán thường gặp là tìm đường đi ngắn nhất trong một mê cung hoặc lưới ô vuông từ điểm xuất phát đến điểm kết thúc. Mê cung có thể được biểu diễn bằng một lưới 2D, với các ô tường và các ô có thể di chuyển. Nhiệm vụ của bạn là tìm cách di chuyển từ điểm xuất phát đến điểm đích sao cho quãng đường ngắn nhất.

Giải pháp

BFS (Breadth-First Search) là một công cụ hiệu quả để giải bài toán này, bởi vì BFS tìm kiếm theo từng lớp (layer-by-layer). Khi gặp điểm đích, BFS đảm bảo rằng đường đi tìm được là ngắn nhất (vì nó khám phá tất cả các đường đi ngắn hơn trước khi khám phá các đường dài hơn).

Cách thực hiện

1. Bắt đầu từ điểm xuất phát và thêm nó vào hàng đợi (queue).
2. Tại mỗi bước, lấy một ô từ hàng đợi và kiểm tra các ô liền kề (lên, xuống, trái, phải).
3. Nếu ô liền kề là điểm đích, thì ta đã tìm được đường đi ngắn nhất.
4. Nếu không, tiếp tục thêm các ô liền kề có thể đi vào hàng đợi để duyệt ở bước tiếp theo.
5. Lặp lại quá trình này cho đến khi tìm thấy đích hoặc duyệt hết lưới.

Mã giả (Pseudocode)

Mã giả thuật toán tìm đường trong mê cung bằng BFS

Input: Ma trận mê cung maze, điểm xuất phát start, điểm đích goal

Output: "Path found" nếu tồn tại đường đi từ start đến goal, ngược lại trả về "No path found"

Function BFS_Maze(maze, start, goal):

```
queue ← ∅
queue.push(start)
visited ← ∅
visited.add(start)
while queue không rỗng do
    current ← queue.pop()
    if current = goal then
        return "Path found"
    foreach neighbor trong các hướng (trên, dưới, trái, phải) do
        if neighbor nằm trong phạm vi hợp lệ and chưa được thăm and không phải là tường then
            queue.push(neighbor)
            visited.add(neighbor)
return "No path found"
```
